# kanhe: multilingual NLI over Swiss voting booklets, with checked quotes

- **Track:** Track 2A, OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Online
- **Team:** kanhe
- **Demo:** `make demo` answers 6 cases (tasks A and B) through the evaluation contract, without an API key; no hosted demo

## 1. Summary

kanhe takes a claim and a Swiss federal voting booklet, each in German, French or Italian in any combination, and answers entailment (0), neutral (1) or contradiction (2), with the booklet sentences it relies on; a program checks that each of them really is in the booklet. It answers both tasks of the challenge through the organisers' contract (`--input cases.jsonl --output predictions.jsonl`). Apertus-v1.5-8B does all the language work: it translates the claim into the booklet's language when the two differ, then judges it against a few paragraphs picked by BM25.

- **Task B** (claim and the matching passage). On the held-out 2025–2026 booklets (331 rows, run once with a configuration fixed on 2020–2024), macro-F1 is **0.967** (95% CI 0.946–0.983) at **923 tokens and 0.85 s per claim**. Giving the model the whole passage instead is statistically level (0.973; paired difference −0.006, CI −0.021 to +0.007) at 2.4 times the tokens.
- **Task A** (claim, the whole booklet PDF and the vote's name). kanhe keeps the booklet's pages about the vote, found with BM25 on the vote's name, then the 8 best paragraphs: **0.913** (CI 0.885–0.941) on test at **1,272 tokens and 0.91 s per claim**. Giving the model the whole PDF instead scores 0.538 on 70 dev rows at 42,495 tokens and 9.05 s per claim, where kanhe scores 0.864 with 3% of the tokens.

96% of task B's and 90% of task A's entailment and contradiction answers carry a quote that a letter-by-letter check finds in the booklet; task A's quotes carry their PDF page. Without the booklet the same model scores 0.27 on dev, but a word-count classifier on the claims reaches 0.61: the claims give part of the label away (§5). `make run` recomputes every table from a cache of the model's answers, without an API key.

## 2. Architecture

Two pipelines share every step except how the context is chosen. **A** (ours) narrows the booklet down before the model reads it; **C** (the baseline) sends everything: the whole passage in task B, the whole PDF in task A.

1. **Read.** Rules split the text into numbered paragraphs (`p01`, `p02`, …) at no token cost: headings and hard-wrapped lines are joined, hyphenated words rejoined, paragraphs over 1,200 characters cut at sentence ends. Task B's passage title is shown to the model as the vote's name. Task A reads the PDF page by page (pypdf), and each paragraph keeps its page number.
2. **Find the vote (task A).** A booklet explains every vote of a voting day, up to 5. BM25 scores each page against the vote's name, and the pages that score at least a third of the best page are kept: 31% of the paragraphs on dev, and in every dev and test row the passage the label was written for (`results/coverage.md`).
3. **Translate (A only).** If the claim's language differs from the booklet's, Apertus translates the claim, keeping names, numbers, dates and amounts unchanged.
4. **Retrieve (A only).** Okapi BM25 (k1 = 1.5, b = 0.75; words lower-cased, accents removed, cut to 6 characters) ranks the paragraphs against the claim. Task B keeps the top 3, plus the passage's first paragraph, which often states the recommendation of the Federal Council and Parliament; task A keeps the top 8.
5. **Judge.** Apertus gets the numbered paragraphs, the vote's name and the claim, and answers with one JSON object: the label, and the sentences it relies on, copied from the excerpt with their paragraph ids. An answer that cannot be read is asked for once more, with the parse error.

6. **Check the quotes.** Each quote must occur in the paragraphs the model was shown. The check compares letters and digits only, so it ignores case, spacing, punctuation and PDF line-break artefacts such as `défini tivement`; numbers must match exactly. "..." marks an omission, and every piece must start and end on a word boundary, so `5 Prozent` does not match inside `15 Prozent`. A verified quote is filed under the paragraph (and page) where it starts.
7. **Apply the rule.** *Lenient* (the default, and the headline): the label stands. *Strict*: an entailment or contradiction without a verified quote goes back to the model once, with the quotes that failed; if it still has none, it becomes neutral.

**The contract.** `python main.py --input cases.jsonl --output predictions.jsonl`, or the Docker image with the same two options, reads the organisers' cases (`id`, `vote`, `claim`, and `booklet` for task A or `reference` for task B) and writes one answer per id: the label, `evidence` as `{page, text}` (1-based PDF pages in task A, `null` in task B) and `metrics` (input and output tokens of all the case's calls, wall-clock milliseconds). A quote that failed the check is replaced by the sentence of the cited paragraph that shares most of its words, so every entailment and contradiction carries verbatim text. Model calls go to `BASE_URL` with `API_KEY`; a case that cannot be read is answered neutral, so every id gets an answer. All calls also go through an on-disk cache keyed by a hash of the model, the messages and the parameters (`data/cache/`), so every result replays without a key. The code is a Python 3.12 package (`src/kanhe/`, 183 unit tests on a scripted fake model) run in Docker; the model runs on the hosted CSCS endpoint. A task A case from `examples/cases.jsonl` (dev row 1475, replayed from the cache, shortened): an Italian claim against the French booklet of 29 November 2020, which explains 2 votes.


```
{"id": "v1.1-row-1475-A", "vote": "Initiative populaire « Pour une interdiction du financement des
producteurs de matériel de guerre »",
 "claim": {"text": "Secondo il testo sottoposto a votazione, l’Assemblea federale raccomanda al popolo e
ai cantoni di approvare l’iniziativa.", "language": "it"},
 "booklet": {"path": "booklets/2020_11_29_fr.pdf", "language": "fr"}}
{"id": "v1.1-row-1475-A", "label": 2, "label_name": "contradiction",
 "evidence": [{"page": 31, "text": "Art. 2 - - L’Assemblée fédérale recommande au peuple et aux cantons de
rejeter l’initiative."}],
 "metrics": {"input_tokens": 961, "output_tokens": 80, ...}}
```


## 3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` for the system. For comparison (task B, dev), `swiss-ai/Apertus-v1.5-70B-thinking` with reasoning switched off (`chat_template_kwargs: {"enable_thinking": false}`); our API key had no access to the plain 70B. No other model is used, neither in the system nor in its evaluation.
- **How it is used:** inference only, zero-shot, wherever the text has to be understood: to translate the claim and to judge it against the booklet. Splitting the booklet, finding the vote's pages and ranking paragraphs are left to rules and BM25, which cost no tokens. No fine-tuning.
- **Where it runs:** the CSCS hosted endpoint (`https://api.inference.cscs.ch/v1/`, OpenAI-compatible chat completions), temperature 0, at most 1,024 output tokens for a judgement and 512 for a translation. The endpoint and key come from `BASE_URL` and `API_KEY`, the names the evaluation sets (or `LLM_BASE_URL` and `LLM_API_KEY`), the model from `LLM_NAME`, by default `swiss-ai/Apertus-v1.5-8B`.

The judge's system prompt:

```
You check claims against the official explanations of the Swiss Federal Council for a federal vote (the
voting booklet).
Decide how the booklet excerpt relates to the claim:
- "entailment": the excerpt states the claim or clearly implies it.
- "contradiction": the excerpt states something that cannot be true together with the claim.
- "neutral": the excerpt does not say enough to decide, or it is about something else.

Rules:
- Use only the excerpt. Do not use outside knowledge and do not judge whether the claim is true in the
real world.
- The claim and the excerpt can be in different languages (German, French, Italian).
- Evidence: copy the sentences you rely on character for character from the excerpt, in the excerpt's
```

```
language, with the id of the paragraph each one comes from. Give no evidence for "neutral".

Answer with one JSON object and nothing else:
{"evidence": [{"id": "p03", "quote": "..."}], "label": "entailment" | "neutral" | "contradiction"}
```

The user message holds `Booklet excerpt, vote: <title>`, one line per paragraph (`[p01] …`) and `Claim: <claim>`. The translation prompt (`src/kanhe/translate.py`) asks for the translation only, with names, numbers, dates and amounts kept as they are; the strict rule's follow-up question and the claim-only prompt are in `src/kanhe/judge.py`.

## 4. Data

- **Benchmark:** `OSTswiss/MNLIoverSwissVotingBooklets` (Hugging Face, MIT), file `v1.1.jsonl`, sha256 `87ee4afd…2fb02fd`: 1,488 rows, labels balanced (495, 498, 495), all 9 claim→booklet language pairs (65% across languages), booklets from 2020 to 2026. Each row's reference is one passage of one vote's text (summary, background, arguments, or the voting text): a title and a body. v1.1 (2026-10-06) is v1.0 without 7 rows whose reference had no text; every number in this report is computed on v1.1.
- **Task A** reads the PDF at each row's `booklet_url`: 60 booklets (45 dev, 15 test), named and turned into cases as the organisers' `prepare_cases.py` does. Their page texts are in `data/booklets.jsonl`, so task A replays without downloading the PDFs.
- **Split by booklet year**, to imitate booklets the system has never seen: *dev*, 2020–2024 (1,157 rows), for every choice we made; *test*, 2025–2026 (331 rows), run once per task with the final configuration.
- **Repeated rows:** some rows repeat word for word, label included. Dev has 897 distinct claim–reference pairs among its 1,157 rows, test 256 among 331; no pair has conflicting labels. We score all rows as published and check every conclusion on distinct pairs too (§5).
- `baseline_score` is not used; the organisers confirmed it is not part of the evaluation input.
- `data/` (27 MB) holds a copy of the benchmark, the booklets' page texts and the cache of model answers. The booklets are official public documents and the claims are about public votes; there is no personal data.

## 5. Evaluation

**Metrics.** Macro-F1 over the 3 labels (checked against scikit-learn's), with a 95% bootstrap interval (1,000 resamples of the rows, seed 0), for each task separately, as the organisers report it. An unreadable answer counts as a miss; no row is ever dropped. Two configurations are compared on the same rows with a paired bootstrap (`kanhe compare`); we call a difference real only when its interval excludes zero, and check it again on distinct claim–reference pairs and by resampling whole booklets (all votes and languages of one voting day: 15 on dev; the 5 on test are too few for this check). Evidence is the share of entailment and contradiction answers with at least 1 verified quote. Token usage per claim ("Tokens / claim"): input plus output tokens of all its calls. Inference time per claim ("Latency / claim"): the sum of its calls' times against the shared endpoint, with 4 claims in flight.

**Table 1. Test split** (2025–2026 booklets, 331 rows, run once per task; Apertus-v1.5-8B). Tokens and latency are for the lenient rule. In task B, A reads 822 input tokens per claim and writes 101, C reads 2,107 and writes 76; the strict rule's follow-up questions bring the totals to 953 (A) and 2,247 (C).

| Task | Setup | Macro-F1, lenient (headline) | Macro-F1, strict | Tokens / claim | Latency / claim |
|---|---|---|---|---|---|
| B | Baseline C: the whole passage | 0.973 [0.954, 0.988] | 0.925 [0.893, 0.952] | 2,183 | 0.72 s |
| B | **Ours A:** translate → BM25 top 3 + first paragraph | **0.967** [0.946, 0.983] | 0.949 [0.924, 0.970] | **923** | 0.85 s |
| B | A − C, paired | −0.006 [−0.021, +0.007] | +0.024 [−0.003, +0.052] | −58% | +18% |
| A | **Ours A:** vote's pages → translate → BM25 top 8 | **0.913** [0.885, 0.941] | | 1,272 | 0.91 s |

In task B, A and C are level under both rules: the paired intervals include zero. A needs 42% of C's tokens. It is slower overall because a cross-language claim (207 of the 331 rows) costs it a second call, the translation:

1.02 s against C's 0.73 s. On same-language claims (124 rows), one call each, A is faster: 0.57 s against 0.71 s. On dev, under the strict rule, A is ahead (+0.019 [+0.007, +0.031]). Scoring only distinct pairs, or resampling whole booklets, leaves these conclusions unchanged (`results/comparisons.md`). In task A, finding the passage in the booklet costs 0.054 against task B on the same rows ([0.022, 0.085], paired). The 8 paragraphs held the passage in 319 of the 331 rows, and in 25 of task A's 31 wrong answers: most of the cost is in judging with more of the booklet in view.

**Table 2. Task A on dev** (2020–2024 booklets). C reads every page of the booklet; it ran on a fixed sample of 70 rows (seed 0) to keep its cost near 3 million tokens.

| Setup | Rows | Macro-F1 | Tokens / claim | Latency / claim |
|---|---|---|---|---|
| Baseline C: whole PDF → Apertus | 70 | 0.538 [0.433, 0.630] | 42,495 | 9.05 s |
| **Ours A**, same 70 rows | 70 | **0.864** [0.784, 0.941] | **1,270** | 0.91 s |
| Ours A with the top 3 paragraphs | 1,157 | 0.915 [0.899, 0.931] | 771 | 0.80 s |
| **Ours A** (top 8), all dev rows | 1,157 | **0.935** [0.921, 0.950] | 1,293 | 1.01 s |
| Ours A on the passage (task B, Table 4) | 1,157 | 0.977 [0.968, 0.986] | 930 | 0.90 s |

This is the comparison the challenge suggests, and on real booklets it is not close: C reads 33 times the tokens, takes 10 times as long and scores 0.326 lower ([0.218, 0.438], paired). With a whole booklet in its context, 8B calls most contradictions neutral (F1 0.200). Finding the vote's pages is what lets A work: among them, BM25's top 8 holds the passage in 97% of dev rows, against 88% when it searches the whole booklet (top 3: 90% against 76%), and 8 paragraphs beat 3 by 0.020 ([0.006, 0.034]). A pays 0.042 on dev for having to find the passage ([0.027, 0.056]).

**Table 3. What the benchmark gives away** (dev split, 1,157 rows; the test split stays untouched).

| Setup | Macro-F1 |
|---|---|
| Uniform guess | ≈ 0.33 |
| Apertus-8B, claim only, no booklet | 0.273 [0.247, 0.297] |
| Naive Bayes on the claim's words, trained on 4 years, tested on the fifth | 0.606 [0.578, 0.633] |
| Ours: task B, A, k = 3, lenient | 0.977 [0.968, 0.986] |

A naive Bayes classifier that never sees the booklet gets 0.61, best on neutral (F1 0.77): neutral claims keep coming back to a few topics ($CO_2$ in 59 of the 389, against 8 of the 768 other claims) and almost never contain a negation word (4 of 389, against 62 of 392 entailments and 76 of 376 contradictions). Given the claim alone, with the prompt saying that the 3 answers are about equally common, our model scores 0.27, below a uniform guess, because it answers neutral 86% of the time. With the booklet it scores 0.985 on the 716 dev rows the classifier gets right and 0.949 on the 441 it gets wrong: those rows are harder, or the wording helps a little; the data cannot tell which (`results/baselines.md`). A system trained on this benchmark could learn the cues, so a high score alone does not show that a system reads.

**Table 4. Task B on dev, all configurations** (1,157 rows; tokens and latency for the lenient rule; 70B with reasoning off).

| Setup | 8B strict | 8B lenient | 8B tokens | 8B latency | 70B strict | 70B lenient | 70B latency |
|---|---|---|---|---|---|---|---|
| C | 0.942 | 0.974 | 2,157 | 0.76 s | 0.904 | 0.931 | 1.56 s |
| A, k = 1 | 0.923 | 0.940 | 720 | 0.79 s | 0.922 | 0.943 | 1.55 s |
| A, k = 2 | 0.961 | 0.978 | 824 | 0.81 s | 0.930 | 0.955 | 1.72 s |
| **A, k = 3** | **0.961** | **0.977** | **930** | **0.90 s** | 0.939 | 0.960 | 1.81 s |
| A, k = 5 | 0.963 | 0.976 | 1,105 | 0.89 s | 0.946 | 0.954 | 1.89 s |
| A, k = 8 | 0.959 | 0.979 | 1,348 | 0.92 s | 0.955 | 0.955 | 1.90 s |

- **k.** In task B the scores plateau from k = 2 on, within their intervals; we kept k = 3, the default before the sweep, to leave retrieval some room on unseen booklets. Task A, with far more paragraphs to choose from, takes k = 8 (Table 2).
- **8B against 70B** (task B). 8B is at least as good as 70B (reasoning off) in 11 of the 12 dev configurations, at half the latency, and ties in the twelfth (A, k = 1, lenient: −0.003 [−0.016, +0.010]).

70B trails most on C (by 0.043 [0.029, 0.058]): it calls 59 of the 389 neutral claims contradictions; 8B does so once. With reasoning on, one 70B judgement took 6.6 s instead of 0.6 s in a probe, so we did not run it.

- **Languages** (test). Task B, by claim language: German 0.971, French 0.959, Italian 0.973; by booklet language: German 1.000, French 0.984, Italian 0.915 [0.855, 0.964], where on dev the Italian booklets score highest (0.992). Task A, by booklet language: German 0.885, French 0.929, Italian 0.917; by claim language, German claims are its weak spot (0.816 [0.740, 0.886], against 0.945 and 0.973).
- **Evidence.** On test, task B's A has a verified quote in 209 of 218 entailment and contradiction answers (95.9%; C: 201 of 219, 91.8%), task A's in 184 of 204 (90.2%). On dev, 85% of task A's verified quotes (972 of 1,147) lie on a page that holds the passage the label was written for. The answers and quotes of Tables 1 and 2 are in `results/<split>/*.predictions.jsonl`, and `make run` writes those of every run.
- **Strict against lenient** (task B). The strict rule costs 0.018 on test ([0.003, 0.036], paired), mostly through copying rather than judging: of the 9 answers it changed, 7 had been right, and most of their quotes had failed on a miscopy such as `BEITE` for `BREITE` or on 2 sentences spliced without an ellipsis. The lenient rule is our headline; the strict rule is for users who need a verified quote behind every decision.

## 6. Limitations

- **Task A trusts the vote's name and the PDF's text.** A page that never names the vote is dropped (no dev or test passage was). Running headers and page numbers stay in the text, and the model sometimes copies them into a quote. 4 test answers of task A could not be read (unescaped quotation marks in the JSON): the scores count them as misses; since the test run, the contract answers such a case with the label it names, which is right for all 4.
- **The benchmark is partly easy** (Table 3), and its references are single passages; task A's cases reuse labels written for a passage, for the whole booklet.
- **The quote check is lexical.** It proves that a sentence is in the booklet, not that it supports the label, and we have not measured support. It rejects quotes of the vote's title, and lets "..." join pieces that lie far apart.
- **8B copies imperfectly**, which is what the strict rule pays for; 70B was run only with reasoning off.
- **Translation costs a call and can slip.** On test, `Id-e-Gesetzgebung` (the e-ID law) came out as `iniziativa legislativa`, and task B's A answered neutral on the 2 rows with that claim, where C, which reads the German claim, was right.
- **One run per configuration**, at temperature 0, on a shared endpoint whose load sets the latency; cache replays are exact but are not new runs, so run-to-run variance is not measured. The first attempt at task A's test run met an overloaded endpoint (5 to 223 s per call); we stopped it before reading any answer, dropped its 6 calls from the cache and ran it again: the 6 answers came back identical.
- **Civic use:** the system reports what the booklet says, not whether a claim is true, and gives no voting advice. The quotes are there so that a reader can check the booklet.

## 7. Reproducibility

- `make run` (from the repository root or `track_2a/`) builds the Docker image (`python:3.12-slim`, uv 0.12.0, locked dependencies) and runs `kanhe reproduce`: it replays the 34 configurations in `configs/report.json` from `data/cache/` without an API key, and prints and writes every table, the paired comparisons, task A's search for the passage and the no-booklet baselines (`results/`) and the curves (`docs/`). In a fresh clone the image built in 40 s (without Docker's layer cache) and the replay took 4.2 minutes, on Docker Desktop (Windows 11, 12 logical CPUs, no GPU, base image already downloaded). The headlines are the tables headed `A-k3-lenient · swiss-ai/Apertus-v1.5-8B · test` (task B) and `A-k8-lenient-pdf · … · test` (task A).
- `make demo` runs the image the way the organisers evaluate a submission, `docker run -v examples:/data:ro -v output:/output <image> --input /data/cases.jsonl --output`

`/output/predictions.jsonl`, here with `--no-live` to replay the 6 cases from the cache (3 s).

- Without Docker: `uv run kanhe reproduce --no-live`; `make test` runs the 183 unit tests offline. With `API_KEY` and `BASE_URL` set (or in `track_2a/.env`, see `.env.example`), calls missing from the cache go to the endpoint and into the cache.
- `reproduce` stops if the dataset's sha256 is not the published one; every summary records the commit it was computed at; temperature is 0; cache keys hash the model, messages and parameters; the bootstrap uses seed 0. A hosted model need not answer bit for bit the same over time, hence the cache.
- The test split was run once per task: task B at commit `4ec9077` (on v1.0; v1.1 drops 3 of its rows, so its scores are recomputed from the same answers), task A with the configuration fixed at `f965116`. The cache holds the answers to all 17,530 calls our experiments made: about 23.2 million input and 1.3 million output tokens.

## 8. Next steps

- **Task A:** drop running headers before judging, tell the judge whose words each paragraph is (the Federal Council's, the committee's), and look into German claims, its weak spot on test.
- **Drop the translation call:** retrieve across languages with multilingual embeddings and give the judge the claim as written (C shows it copes with mixed languages), to win back A's latency and avoid translation slips.
- **Exact quotes from 8B:** quote by sentence id, or constrain decoding to the excerpt, so that the strict rule costs nothing; then send only answers without a verified quote to 70B with reasoning.
- **A harder benchmark slice:** neutral claims from the same vote and contradictions beyond flipped recommendations; we would share the claim-only findings with the benchmark's authors.

## License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. The code is under Apache-2.0 (`LICENSE`); our cache of model answers (`data/cache/`) under CDLA-Permissive-2.0. The copy of the benchmark keeps its MIT licence; the booklets are official publications of the Swiss Federal Chancellery.

## References

- Swiss AI Initiative. Apertus technical report. https://github.com/swiss-ai/apertus-tech-report
- OST. Multilingual NLI over Swiss Official Voting Booklets (dataset). https://huggingface.co/datasets/OSTswiss/MNLIoverSwissVotingBooklets
- Swiss Federal Chancellery. Sammlung der Abstimmungsbüchlein seit 1978. https://www.bk.admin.ch/de/sammlung-der-abstimmungsbuechlein-seit-1978
- Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3(4).
- Gururangan, S., Swayamdipta, S., Levy, O., Schwartz, R., Bowman, S. R. and Smith, N. A. (2018). Annotation Artifacts in Natural Language Inference Data. *NAACL*.
- Poliak, A., Naradowsky, J., Haldar, A., Rudinger, R. and Van Durme, B. (2018). Hypothesis Only Baselines in Natural Language Inference. **SEM*.
- Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.